Bayes' theorem

$x = (x_1, x_2, \ldots x_n)$

$P(x_1, x_2, \ldots x_n | y) =$ (Classifier)

given y, all are ind. $\Rightarrow \prod_{i=1}^{n} P(x_i, y)$

(Naive assumption)

Q) New x ?

GDA :-

Gaussian classifier

x follows gaussian.

$x | y=0 \sim \mathcal{N}(\mu_0, \Sigma)$

$x | y=1 \sim \mathcal{N}(\mu_1, \Sigma)$

ϕ is related to how long the centres (means) are

$\phi = 0.5$

~~Posterior prob~~

Predict $y=1$ if $g_1(x) > g_0(x)$.

$-\frac{1}{2}(x-\mu_1)^T \Sigma^{-1}(x-\mu_1) + \log\phi >$

$-\frac{1}{2}(x-\mu_0)^T \Sigma^{-1}(x-\mu_0) + \log(1-\phi)$

$x^T \Sigma^{-1} x - 2\mu_1^T \Sigma^{-1} x + \mu_1^T \Sigma^{-1} \mu_1$

$w^T x + b = 0$ where $w = \Sigma^{-1}(\mu_1 - \mu_0)$

$y = \begin{cases} 1 & w^T x + b > 0 \\ 0 & w^T x + b \le 0 \end{cases}$

hyperplane

Basically $p(x|0) > p(x|1)$ iff

Classification Rule $\Rightarrow$ $\|x - \mu_0\| < \|x - \mu_1\|$ (= at decision boundary)

Q) $x | y=0 \sim \mathcal{N}(\mu_0, \Sigma)$

$x | y=1 \sim \mathcal{N}(\mu_1, \Sigma)$

$\mu_0 \neq \mu_1$.

$y \in \{0, 1\}$.

During training, we estimate $\mu_0, \mu_1, \Sigma, \phi$

$P(y=1) = \phi$

$P(y=0) = 1 - \phi$

$P(y|x) \propto P(x|y) \cdot P(y)$

Find predicted y for new x

$P(y=y|x) = \frac{P(x|y=y)\, P(y=y)}{P(x)}$

$\hat{y} = \arg\max_y P(x|y=y) \cdot P(y=y)$

$$P(x|y=y) = \frac{1}{(2\pi)^{d/2} |\Sigma|^{1/2}} e^{-\frac{1}{2}(x-\mu_y)^T \Sigma^{-1} (x-\mu_y)}$$

for class 0

$P(x|y=0) \cdot P(y=0) = (e^{-\frac{1}{2}(x-\mu_0)^T \Sigma^{-1}(x-\mu_0)} (1-\phi)$

for class 1

$P(x|y=1) \cdot P(y=1) = (e^{-1/2(x-\mu_1)^T \Sigma^{-1}(x-\mu_1)} \phi$

$g_y(x) = -\frac{1}{2}(x-\mu_y)^T \Sigma^{-1}(x-\mu_y) + \log P(y=y)$

$\hat{y} = \arg\max_y g_y(x)$.

$g_0(x) = -\frac{1}{2}(x-\mu_0)^T \Sigma^{-1}(x-\mu_0) + \log(1-\phi)$

$g_1(x) = -\frac{1}{2}(x-\mu_1)^T \Sigma^{-1}(x-\mu_1) + \log\phi$

6/4/26

11 (2)28

K means clustering

(assign points → move centers
repeat).

$$J = \sum\sum r_{nk} \| x_n - \mu_k \|^2$$

r_{nk}: 1 if $n \in$ cluster k, 0.

x_n: data point. μ_k: centroid

(New centroid = avg of all pts belonging to that cluster).

Alternating optimization

(Elbow finding)

Iterative process of K means. K means finds the optimal way to which centroid is closer.

choose centroids → assign pts to nearest centroid

move centroid to mean

Probabilistic clustering

(unsupervised learning)

$D_x = \{x_n\}_{n=1}^{N}$ (we don't have y)

→ As we can't model $P(x|y)$ so we work $P(x)$

Idea: Modelling $P(x)$ (marginal likelihood) ⇒ Gaussian Mixture Models (GMMs)

prob of coming from Gaussian 1, 2, etc.

D>N couples (x_1, y_n)

Dr CD {distance only y's}

$$P_x = \pi_1 P(x|z=1) + \pi_2 P(x|z=2)$$

In GMM, the distributions are Gaussian.

$$P(x) = \sum_{k=1}^{K} \pi_k N(x|\mu_k, \Sigma_k)$$

$$P_x = \sum_{k=1}^{K} P(x|z=k) P(z=k)$$

Law of diminishing returns ⇒ after benefit is decreasing adding clusters helps but elbow method

2D ⇒ 2 dimensional vector of gaussians

$$x = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} \quad x|z=k \sim N(\mu_k, \Sigma_k)$$

3D: $h_t = f(x)$.

Multiple clusters produce multiple gaussian hills..

$$P(x) = \underbrace{\pi_1 N_1(x)}_{\text{hill 1}} + \underbrace{\pi_2 N_2(x)}_{\text{hill 2}} + \underbrace{\pi_3 N_3(x)}_{\text{hill 3}}$$

soft/ probabilistic assignment.

Oct 10 28

ML Class

Jenson inequality

log ⇒ concave, ↑ gives

$E[f(x)] \leq f(E[x])$

∴ for log

$E[\log x] \leq \log(E[x])$

avg ↑ log ⇒ ↑ value

general Jensen:

$\alpha f(a) + (1-\alpha) f(b) \geq f(\alpha a + (1-\alpha) b)$

(for convex).

$E[\log(U)] \leq \log(E[U])$.

$\sum_z Q(z) \log \frac{P(x, z, \omega)}{Q(z)} \leq \log \sum_z Q(z) \frac{P(x, z, \omega)}{Q(z)}$

Evidence lower Bound (ELBO).

$\therefore E[\log U] \leq \log(E[U])$.

$U = \frac{p(x, z, \omega)}{q(z)}$.

$E_{q(z)}\left[\log \frac{p(x, z, \omega)}{q(z)}\right] \leq \log E_{q(z)}\left[\frac{p(x, z, \omega)}{q(z)}\right]$

$\because E_{q(z)}\left[\frac{p(x, z, \omega)}{q(z)}\right] = \sum_z q(z) \frac{p(x, z, \omega)}{q(z)}$

$\Downarrow$ summing over z

$p(x, \omega)$.

$\therefore E_{q(z)}\left[\log \frac{p(x, z, \omega)}{q(z)}\right] \leq \log p(x, \omega)$.

$\alpha(\omega)$.

$\therefore \boxed{\alpha(\omega) \leq \log p(x, \omega)}$

$\log p(x^i, \omega) = \log \sum_{k=1}^{K} \pi_k \mathcal{N}(x^i, \mu_k, \Sigma_k)$

EM type optimisation:

$\log p(x, \omega) \geq ELBO(x, Q, \omega)$.

$ELBO(x, Q, \omega) = \sum_z Q(z) \log \frac{p(x, z, \omega)}{Q(z)}$.

x = observed data

z = latent variable

$Q(z)$ = chosen distribution

Expectation - Maximization

(1) Set $Q(z)$
(2) Maximize $ELBO(x, Q, w)$.

where maximizing ELBO.

E step :- $\frac{p(x, z; w)}{Q(z)} \rightarrow p(z|x, w)$

once $Q(z)$ is set,

M step :- $\max_w ELBO(x, Q, w)$

E step : choose / update $Q(z)$
M step : optimize w.

(repeat)

original goal : $\max_w \log p(x; w)$

13/10/26

Decision Trees

Heart Disease dataset.

input output goal.

(इसमें tree में features के आधार पर Patient में heart disease है या नहीं).

Tree में हम question के data को smaller groups में divide करते हैं।

Entropy: measuring confusion / impurity

(Completely pure group) ← 0 ... 1 → (maximum confusion)

यहां 1 से 0 की ओर

Infogain: by Age>50, weighted entropy.

One hot mentioned.

$$IG = H_{before} - H_{after}$$

Decision Tree algo:

Current dataset

Try diff features

Calculate IG

Calculate highest IG

Repeat recursively (until stopping condition met)

Data → multiple versions created

Bootstrapping:

Dataset D → N data sets.

Sample w/ replacement → original dataset of 5 patients randomly, & put patient back after selection.

original dataset: A, B, C, D, E

Bootstrap sample 1: B B E A D

" sample 2: A C C E B

} repetitions allowed

What's ensemble learning

↓

multiple models to combine.

Idea:

original dataset D

Generate bootstrap samples

S1 S2 SR

Tree1 Tree2 TreeR

Combine Predictions

Random Forest

∴ Multiple decision trees. ∵ Sample of tree train.

Ensemble: Tree predictions combine.

DOMS

(Random forest, Gradient boosting → Confusion Performance)

Q

$$D = \begin{bmatrix} 4.3 & 2.4 \\ 2.1 & 7.1 \\ 5.3 & 2.3 \end{bmatrix}$$

original dataset D → D', D'', D'''

(random sampling w/ replacement)

Bootstrap Aggregation (Bagging)

Algo to combine decision trees.

Problem: High variance.

Soln: Train multiple trees instead of one & combine their predictions.

$$f_{bag}(x) = \frac{1}{B}\sum_{b=1}^{B} f_b(x)$$

B: Total trees; $f_b(x)$: Pred. of tree b; x: input

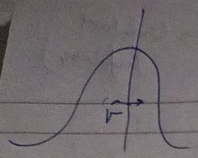
Q

Tree	Pred. Score
1	0.8
2	0.6
3	0.7

$f_{bag}(x) = \frac{0.8+0.6+0.7}{3} = 0.7$

∴ Bagging is use full for high variance - models.

DOMS



Out-of-Bag (OOB) Probability

Ex In D, N=100 obs. → train a decision tree.
∴ sampling w/ replacement, and we select observations 100 times

In-bag :: OOB: * observations, जो bootstrap sample में चुने गए
जो नहीं चुने गए.

Ex A B C D E (5 obs).

bootstrap selection: A D B D A.

In bag: A, B, D. — OOB: C, E.

Prob derivation

$X = \{x^1, x^2, \ldots x^N\}$. $x^i \Rightarrow$ observation.

bootstrap dataset → X_b.

(Ex) $P(x_b = x^i) = \frac{1}{N}$. {random, iid}.

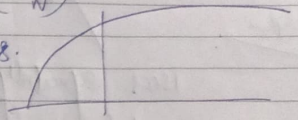
$P(x_b \neq x^i) = 1 - \frac{1}{N}$

for N draws,

$P(x^i \text{ is OOB}) = \left(1 - \frac{1}{N}\right)^N$.

$f(N) = \left(1 - \frac{1}{N}\right)^N$.

$\lim_{N \to \infty} \left(1 - \frac{1}{N}\right)^N = \frac{1}{e} \approx 0.368$.



↳ Dataset मध्ये जी एकदा ही bootstrap sample में grand 36.8% unique observations. असतो सोबत selected नसतो.

(cont') w/ Random forest

एक tree → trained on its bootstrap dataset. त्या obs. या tree ने नाही ते OOB झाले.
जेव्हा test Prediction करायचे model ला असेल तेव्हा

Q. कोणत्या feature Prediction ला [illegible] imp आहे?

Problem: bagging has multiple trees averaged. & it becomes difficult to explain individual tree. we want to know

$\Delta I = I(\text{parent}) - \left[\frac{N_L}{N} I(L) + \frac{N_R}{N} I(R)\right]$

impurity before split. — left child impurity — right child impurity — child nodes मध्ये किती observations.

Boosting for Regression Trees

Algorithm :

Trees sequentially train ~~with~~ 2nd

$f(x) = 0$, $r_i = y_i$ initial pred. (Simple st value)

↓

Tree 1 (initial errors at learn)

current prediction. / actual output

Tree 2 (updated residuals :)

residual.

$r_i = y_i - 0$ Tree 3, 4 (gradually inform).

15/10/26

ML Class

(1) Boosting for Regression Trees — pseudocode

idea

multiple small trees sequentially

$T_1 \rightarrow$ initial predictions, mistakes corrected.

$T_2 \rightarrow$

$T_3 \rightarrow$

$\vdots$

$\vdots$

$\hat{f}(x) = 0, \quad r_i = y_i$

no model.

residual.

in regression, $r_i = y_i - \hat{f}(x_i)$

fit a tree on (X, r)

repeat for $b = 1, 2, \ldots B$ (total trees)

Fit a tree $\hat{f}^b$ with d splits to (X, r).

$$\hat{f}(x) \leftarrow \hat{f}(x) + \lambda \hat{f}^b(x) \quad \text{— (1)}$$

update.

learning rate / shrinkage parameter.

new prediction.

$$r_i \leftarrow r_i - \lambda \hat{f}^b(x_i) \quad \text{— (2)}$$

Combine all trees

$$\hat{f}(x) = \sum_{b=1}^{B} \lambda \hat{f}^b(x)$$

$$= \lambda \hat{f}^1(x) + \lambda \hat{f}^2(x) + \ldots$$

Ada Boost - M1 (Ensemble learning)

→ Σ ची classifier train करते वेळी difficult to classify data pts ला जास्त importance दिली जाते Σ !

Eg Spam / Not Spam. (Binary classifier).

Ada Boost → 4 वेळा जास्त फसला

1	S
2	NS
3	S
4	NS ✗
5	S

Sol: assign every training example a weight

(1) $w_i = \frac{1}{N}$, $i = 1, \ldots, N$ → total training examples

(equal imp initially).

(2) repeat for $m = 1$ to M → no of classifiers to train

2.a) Fit a classifier $G_m(x)$ using weights w_i.

classifier trained in iteration m.

More wt → mis classify is more costly to training objective.

2.b) $err_m = \frac{\sum_{i=1}^{N} w_i \, I(y_i \neq G_m(x_i))}{\sum_{i=1}^{N} w_i}$

y_i → actual class of example, $G_m(x_i)$ → predicted class of classifier

$I(y \neq G_m(x_i)) = \begin{cases} 1, & \text{Prediction wrong} \\ 0, & \text{Prediction correct} \end{cases}$

Contribution zero.

Eg 5 ex → 0.2 wt. 2 classifier ex 2 & 4 wrong predict

$err_m = \frac{0.2 + 0.2}{0.2 + 0.2 + \ldots 0.2}$

$err_m = \frac{0.4}{1} = 0.4$ (weighted classification error 40%).

(2.c) $\alpha_m = \log\left(\frac{1 - err_m}{err_m}\right)$

G_m चा वापरायचा importance in final ensemble.

Eg $err_m = 0.2$

$\alpha_m = \log\left(\frac{1-0.2}{0.2}\right) = \log 4$

≈ 1.386.

if $err_m = 0.5$, $\alpha_m = 0$

$err_m = 0.6$, $\alpha_m = \log(4/6) < 0$.

Std Ada boost M1 assumes multiclass weak learner whose weighted error is below random guessing threshold

cl. error	cl. imp
Low error	High value
$err = 0.5$	$\alpha_m \approx$
$err > 0.5$	-ve α_m

(2d)

$$w_i \leftarrow w_i \, e^{\alpha_m \cdot I(y_i \neq G_m(x_i))}$$

If $I(y_i \neq G_m(x_i)) = 0$,

$w_i \leftarrow w_i \, e^{0} = w_i$.

If $\quad\quad = 1$,

$w_i \leftarrow w_i \, e^{\alpha_m}$

$\therefore \alpha_m = \log\left(\frac{1-err_m}{err_m}\right)$

$$w_i \leftarrow w_i \left(\frac{1-err_m}{err_m}\right)$$

$\sum_{i=1}^{N} w_i \leftarrow$ normalizing factor too

$$G(x) = \text{sign}\left[\sum_{m=1}^{M} \alpha_m G_m(x)\right]$$

Class fn	Pred^n	Imp
a_1	+1	1.5
a_2	−1	0.5
a_3	+1	2.0

$\phi = 1.5 \times (+1) + 0.5 \times (-1) + 2 \times (+1)$

$= 3$

$\therefore \phi > 0, \quad G(x) = +1$

* Surrogate loss mentioned

Trees, decision trees, Greedy approach

DT are decision structures.

Boosting trees, forward stagewise Additive Modeling

General framework of boosting

$$T(x_i, \Theta) = \sum_{j=1}^{J} \gamma_j I(x \in R_j)$$

decrease decisions of every node. we don't gain them in costs nothing like gradient descent etc

A DT feature space with diff regions & divide & conq

$\Downarrow$

& house prices.

Pred. Price

R_1: $A < 1000$ sqft → 30 lakh

R_2: $A \in [1000, 2000]$ → 60 lakh

R_3: $A > 2000$ sqft → 1 crore

R_j → feature space of jth region.

γ_j = region of region of Prediction.

$$I(x \in R_j) = \begin{cases} 1, & x \text{ belongs to } R_j \\ 0, & \text{otw} \end{cases}$$

By using house R_2 of size x,

$T(x_i, \Theta) = 30(0) + 60(1) + 100(0) = 60$ lakh

$\Theta = \{R_j, \gamma_j\}_{j=1}^{J}$.

Prediction का loss.

How is Tree trained?

actual

$$\hat{\theta} = \arg\min_{\theta} \sum_{j=1}^{J} \sum_{x_i \in R_j} L(y_i, \gamma_j)$$

दिये हुए tree कैसे choose करें जो training data पर total loss minimize करे

L can be anything like, $(y_i - \gamma_j)^2$ as objective accordingly

Combinatorial Problem ⇒ Soln is greedy Recursive splitting.

(1) Best split.
(2) Data के resulting regions पर further split
(3) Repeat until stopping criteria.

$$f_M(x) = \sum_{m=1}^{M} T(x; \theta_m)$$

i.e. $f_3(x) = T_1(x) + T_2(x) + T_3(x)$.

Forward stagewise additive modelling

$f_0(x) = 0$ for $(m = 1$ to $M)$ {

$$(\beta_m, \gamma_m) = \arg\min_{\beta,\gamma} \sum_{i=1} L(y_i, f_{m-1}(x) + \beta b(x;\gamma))$$

Combined prediction. Contribution. Tree

}

New model = Old + New estimate

$$f_m(x) = f_{m-1}(x) + \beta_m b(x, \gamma_m).$$

In Regression boosting,

$$f_m = f_{m-1}(x) + \lambda T_m(x).$$

AdaBoost, we train classifiers & update the weights.

Sequential learning

Why small trees work well & how Grad. Boosting builds them)

Boosting results in highest improvement when we use weak learners

Test error

No. of trees

Large DT can reduce training error but often overfit training data).

Decision stump is a tree w/ one split, 2 terminal leaves

Stumps: 2 leaf trees

Any split is better than random guessing: weak learner!

$f_M(x) = T_1(x) + \dots T_M(x)$ ← regression for each tree

* Weak learners allow gradual improvements.
* small trees can form a powerful model

Gradient Tree Boosting Algo

(building boosted trees using gradients).

$f_0(x) = \arg\min_\gamma \sum_{i=1}^{N} \alpha(y_i, \gamma)$.

$\gamma_{im} = -\left[\frac{\partial \alpha(y_i, f(x_i))}{\partial f(x_i)}\right]_{f=f_{m-1}}$ (pseudo-residuals)

* $\alpha(y_i, f(x_i)) = \frac{1}{2}(y_i - f(x_i))^2$

$\frac{\partial \alpha}{\partial f(x_i)} = f(x_i) - y_i$

$\gamma_{im} = y_i - f(x_i)$.

∴ Pseudo residual = Actual − Current Prediction

Fit a regression tree now, say $T_m(x)$.

$R_{1m}, R_{2m}, \dots R_{Jm}$.

$\gamma_{jm} = \arg\min_\gamma \sum_{x_i \in R_{jm}} \alpha(y_i, f_{m-1}(x_i) + \gamma)$.

* say a leaf has 2 obs

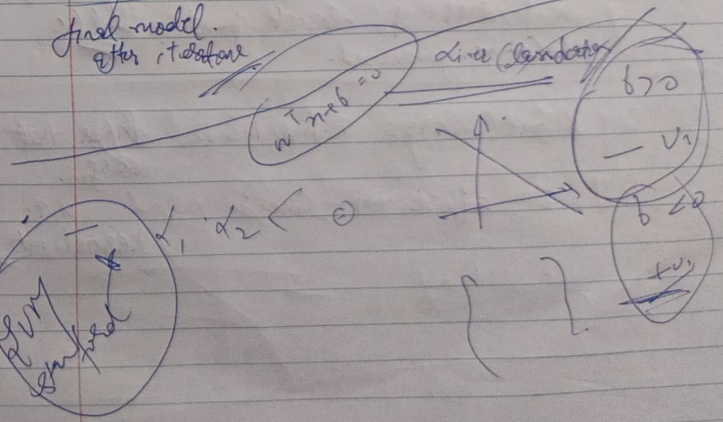
Obs	Act	Curr Pred^n	
A	50	40	$\gamma_A = 10$
B	70	40	$\gamma_B = 20$.

$\gamma_{jm} = \frac{10+20}{2} = 20$. (corrected) (sp. Obs)

∴ updated pred^n = 40 + 20 = 60. for both.

$f_m(x) = f_{m-1}(x) + \lambda T_m(x)$ ← newly trained tree

$f_M(x) = f_0(x) + \lambda \sum_{m=1}^{M} T_m(x)$ ← final model after iterations



16/10/24

ML class

$w = \begin{bmatrix} w_1 \\ w_2 \end{bmatrix}$, $x = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$

$w^T x + b = w_1 x_1 + w_2 x_2 + b$

(Geometric interpretation of Logistic Regression)

Ex $2x_1 - x_2 + 3 = 0$ (decision boundary is linear)

$\vec{w} \perp^r$ to decision boundary.

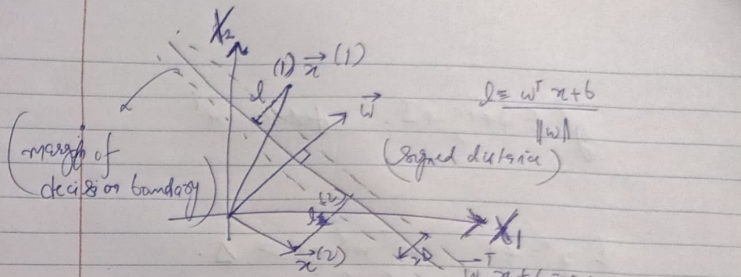
$\sigma(w^T x + b) = \frac{1}{1 + e^{-(w^T x + b)}}$, $P(Y=1 \mid X=x) = \sigma(w^T x + b)$

$d = \frac{w^T x + b}{\|w\|}$

We need to learn the decision boundary which is farther. So closest point should be far. Can be generalised.

SVM define margin around D.Bnd & maximise that margin.

SVM also supports non-linear D.Bnd. using Kernelization



$d^{(1)} = \frac{w^T x^{(1)} + b}{\|w\|}$, $d^{(2)} = -\left(\frac{w^T x^{(2)} + b}{\|w\|}\right)$

$d^{(i)} = y^{(i)} \cdot \left(\frac{w^T x^{(i)} + b}{\|w\|}\right)$

$\frac{(w^T x + b - \Delta)}{\|w\|}$, $\frac{(w^T x + b + \Delta)}{\|w\|}$

Dataset is binary classification. So we want to maximise the margin to correctly classify.

SVM!!

optimization problem

$$\max_{w,b} 2\Delta \quad \text{s.t.} \quad y^{(i)}\left(w^T x^{(i)} + b\right) \geq \Delta, \quad i = 1, \dots N.$$

(Convert) $\|w\|_2^2 = 1$

There will be unique line w/ max margin.

Linearly separable data & maximizing the margin => UNIQUE Soln

$y^i \in \{+1, -1\}$. features $= x^i$

if $y^i = +1$, $w^T x^i + b \geq \Delta$.

$= -1$, $-(w^T x^i + b) \geq \Delta$

$w^T x_i + b \leq -\Delta$.

$\therefore \|w\|_2^2 = 1$, $w_1^2 + w_2^2 + \cdots w_d^2 = 1$

$\therefore d = \frac{w^T x + b}{\|w\|_2} = \frac{w^T x + b}{1}$

($w^T x + b \rightarrow$ affine fn)
if we remove the offset, it becomes affine.

(OO Fancy)

$f(\theta x + (1-\theta) y) \leq \theta f(x) + (1-\theta) f(y)$.

→ Taylor 1st 2nd order
→ subderivative.

(Feasible solution)

